Data Example: Mosaic Pattern II

Spatial Neighbors and Tests

Mahedi Hasan and Jun Zhu

Spatial Neighbors and Lists

In this demonstration, we will be using the small mosaic pattern II from lecture to explore neighborhood structure objects.

II.

B	B	W	W
B	B	B	W
W	B	W	W

Figure 1: Mosaic Pattern 2

First, we load the R libraries we will need. Remember if you haven't installed the packages yet, you need to install the packages first (for example `install.packages("spdep")`).

```
library(spdep)
library(geoR)
```

In this example, let 1 = black and 0 = white - think of a checkerboard. On this checkerboard, we want to first identify who is neighbors with whom.

First, we'll enter the data vector row-wise and call it `egII`.

```
# Enter data vector (row-wise)
# 1 = black; 0 = white
egII = c(1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0)
```

To check that the data are loaded correctly, we can display it as in Figure 1 using the R function `matrix()` and specifying the argument `byrow=TRUE`:

```
matrix(egII, ncol = 4, byrow = TRUE)
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    1    0    0
## [2,]    1    1    1    0
## [3,]    0    1    0    0
```

Remember, 1's are black, and 0's are white. The data entry is indeed correct.

Next, we will apply the function `cell2nb()` from the `spdep` package to `egII`. In particular, we will generate a neighborhood structure object for this grid of cells with 3 rows and 4 columns based on first-order neighbors (argument `type="rook"`).

```
# Generate neighborhood structure object for grid cells
egII.nb <- cell2nb(nrow = 3, ncol = 4, type = "rook")
```

We apply the function `class()` to `egII.nb` and see that the object `egII.nb` is of class `nb`, meaning it is a neighborhood structure object.

```
class(egII.nb)
```

```
## [1] "nb"
```

```
egII.nb
```

```
## Neighbour list object:
## Number of regions: 12
## Number of nonzero links: 34
## Percentage nonzero weights: 23.61111
## Average number of links: 2.833333
```

It also shows us that there are 12 regions (the 12 cells), 34 non-zero links, percentage nonzero weights (23.61%), and the average number of links for a cell (~2.83).

Next, we apply the function `str()` to `egII.nb`.

```
str(egII.nb)
```

```
## List of 12
##  $ : int [1:2] 2 5
##  $ : int [1:3] 1 3 6
##  $ : int [1:3] 2 4 7
##  $ : int [1:2] 3 8
##  $ : int [1:3] 1 6 9
##  $ : int [1:4] 2 5 7 10
##  $ : int [1:4] 3 6 8 11
##  $ : int [1:3] 4 7 12
##  $ : int [1:2] 5 10
##  $ : int [1:3] 6 9 11
##  $ : int [1:3] 7 10 12
```

```
##  $ : int [1:2] 8 11
##  - attr(*, "class")= chr "nb"
##  - attr(*, "call")= language cell2nb(nrow = 3, ncol = 4, type = "rook")
##  - attr(*, "region.id")= chr [1:12] "1:1" "2:1" "3:1" "4:1" ...
##  - attr(*, "cell")= logi TRUE
##  - attr(*, "rook")= logi TRUE
##  - attr(*, "sym")= logi TRUE
```

The structure of `egII.nb` above shows a vector of entries for each of the 12 cells, and the entries for that vector show the indexes of neighbors. Recall, the cell id's are stored as columns, shown by the R function `matrix(seq(1:12), ncol=4, byrow=TRUE)`.

```
matrix(seq(1:12), ncol = 4, byrow = TRUE)
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    2    3    4
## [2,]    5    6    7    8
## [3,]    9   10   11   12
```

For example, for the entry in the first row of the list (cell 1), its first-order neighbors are cells 2 and 5.

Next, we summarize the neighborhood structure object.

```
summary(egII.nb)
```

```
## Neighbour list object:
## Number of regions: 12
## Number of nonzero links: 34
## Percentage nonzero weights: 23.61111
## Average number of links: 2.833333
## Link number distribution:
##
## 2 3 4
## 4 6 2
## 4 least connected regions:
## 1:1 4:1 1:3 4:3 with 2 links
## 2 most connected regions:
## 2:2 3:2 with 4 links
```

The summary of `egII.nb` above gives us more details than what we obtained from just calling the object. We also get information on the most and least connected cells (noted as regions above).

So far we've considered *first-order neighbors* (rook's case). To consider *second-order neighbors* (queen's case), we specify `type=queen` in the `cell2nb()` function.

```
str(cell2nb(nrow = 3, ncol = 4, type = "queen"))
```

```
## List of 12
##  $ : int [1:3] 2 5 6
##  $ : int [1:5] 1 3 5 6 7
##  $ : int [1:5] 2 4 6 7 8
```

```
##  $ : int [1:3] 3 7 8
##  $ : int [1:5] 1 2 6 9 10
##  $ : int [1:8] 1 2 3 5 7 9 10 11
##  $ : int [1:8] 2 3 4 6 8 10 11 12
##  $ : int [1:5] 3 4 7 11 12
##  $ : int [1:3] 5 6 10
##  $ : int [1:5] 5 6 7 9 11
##  $ : int [1:5] 6 7 8 10 12
##  $ : int [1:3] 7 8 11
##  - attr(*, "class")= chr "nb"
##  - attr(*, "call")= language cell2nb(nrow = 3, ncol = 4, type = "queen")
##  - attr(*, "region.id")= chr [1:12] "1:1" "2:1" "3:1" "4:1" ...
##  - attr(*, "cell")= logi TRUE
##  - attr(*, "queen")= logi TRUE
##  - attr(*, "sym")= logi TRUE
```

What are some of the differences you notice in the lists between the first and second-order neighbors? That's right - in the second order, each cell has more neighbors in the neighborhood structure object. For example, for cell 1 in the first row of the list, its second-order neighbors are cells 2, 5, and 6. The additional neighbor is 6, which is a diagonal neighbor of cell 1 in the 3 by 4 grid.

Writing to GAL File

We can save the neighborhood structure object to a text file and then read it back in. The function `write.nb.gal()` will output a neighborhood structure object as a GAL file; `read.gal()` will conversely read the lattice in a GAL file into a neighborhood structure object in R. Both functions are from the `spdep` package.

```
# Output a neighborhood structure object as a GAL lattice file
write.nb.gal(egII.nb, "egIInb.txt")
```

```
# Or, read a GAL lattice file into a neighborhood structure object
args(read.gal)
```

```
## function (file, region.id = NULL, override.id = FALSE)
## NULL
```

```
egII.nb2 = read.gal("egIInb.txt")
```

Spatial Weights for Neighborhood Structure Objects

In this part, we will be using the small mosaic pattern II from lecture to explore spatial weights.

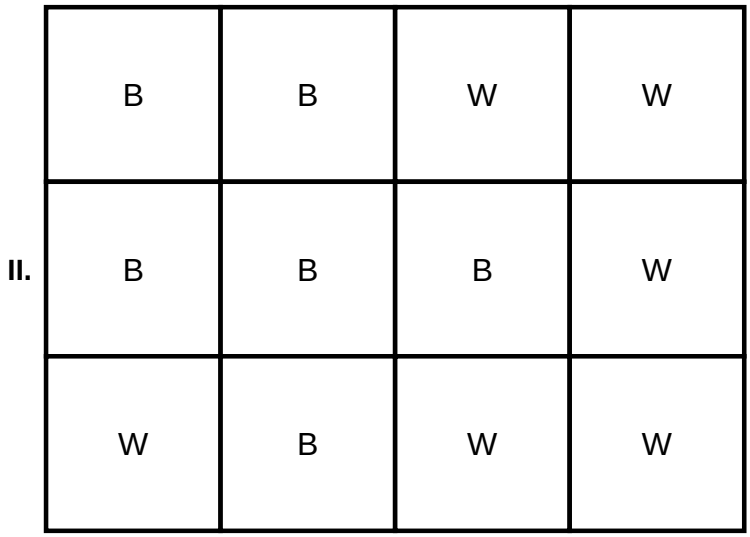


Figure 1: Mosaic Pattern 2

We continue with the mosaic pattern II and consider first-order neighbors. Recall the neighborhood structure object `egII.nb` with the following structure.

```
str(egII.nb)
```

```
## List of 12
##  $ : int [1:2] 2 5
##  $ : int [1:3] 1 3 6
##  $ : int [1:3] 2 4 7
##  $ : int [1:2] 3 8
##  $ : int [1:3] 1 6 9
##  $ : int [1:4] 2 5 7 10
##  $ : int [1:4] 3 6 8 11
##  $ : int [1:3] 4 7 12
##  $ : int [1:2] 5 10
##  $ : int [1:3] 6 9 11
##  $ : int [1:3] 7 10 12
##  $ : int [1:2] 8 11
##  - attr(*, "class")= chr "nb"
##  - attr(*, "call")= language cell2nb(nrow = 3, ncol = 4, type = "rook")
##  - attr(*, "region.id")= chr [1:12] "1:1" "2:1" "3:1" "4:1" ...
##  - attr(*, "cell")= logi TRUE
##  - attr(*, "rook")= logi TRUE
##  - attr(*, "sym")= logi TRUE
```

For the neighborhood structure object above, we now consider assigning spatial weights.

The function we use for assigning spatial weights is the `nb2listw()` function (from the `spdep` package). This function is applied to a neighborhood structure object and different spatial weights can be selected. The output of the function is a list of spatial weights.

The default spatial weight is `W` (argument `style="W"`), which requires that the sum of spatial weights

is 1 for each given cell. That is, the spatial weights are row standardized.

```
# This takes in the neighborhood structure object we created above
egII.lw <- nb2listw(egII.nb)
```

We can inspect the row-standardized spatial weights by looking at the weights component of the list object.

```
egII.lw$weights
```

```
## [[1]]
## [1] 0.5 0.5
##
## [[2]]
## [1] 0.3333333 0.3333333 0.3333333
##
## [[3]]
## [1] 0.3333333 0.3333333 0.3333333
##
## [[4]]
## [1] 0.5 0.5
##
## [[5]]
## [1] 0.3333333 0.3333333 0.3333333
##
## [[6]]
## [1] 0.25 0.25 0.25 0.25
##
## [[7]]
## [1] 0.25 0.25 0.25 0.25
##
## [[8]]
## [1] 0.3333333 0.3333333 0.3333333
##
## [[9]]
## [1] 0.5 0.5
##
## [[10]]
## [1] 0.3333333 0.3333333 0.3333333
##
## [[11]]
## [1] 0.3333333 0.3333333 0.3333333
##
## [[12]]
## [1] 0.5 0.5
##
## attr(,"mode")
## [1] "binary"
## attr(,"W")
```

```
## [1] TRUE
## attr(,"comp")
## attr(,"comp")$d
##  [1] 2 3 3 2 3 4 4 3 2 3 3 2
```

Notice how the spatial weights in each row of the output list sum to 1. For example, for cell 1 its neighbors are 2 and 5, and we know this from `egII.nb`, so each will have a spatial weight of 0.5. Intuitively, this means that both neighboring cells contribute equally in their relationship to cell 1 as its first-order neighbors.

What are some other ways you can think of to assign spatial weights to neighbors? We will look at some alternative spatial weight specifications in R.

Binary Coding

Setting the style argument to `B` gives binary weights (0/1). The weights are 1 if the two cells are neighbors and zero otherwise. Only those neighbors with weight 1 are listed in the output list.

```
# B = basic binary coding
nb2listw(egII.nb, style = "B")$weights
```

```
## [[1]]
## [1] 1 1
##
## [[2]]
## [1] 1 1 1
##
## [[3]]
## [1] 1 1 1
##
## [[4]]
## [1] 1 1
##
## [[5]]
## [1] 1 1 1
##
## [[6]]
## [1] 1 1 1 1
##
## [[7]]
## [1] 1 1 1 1
##
## [[8]]
## [1] 1 1 1
##
## [[9]]
## [1] 1 1
##
## [[10]]
```

```
## [1] 1 1 1
##
## [[11]]
## [1] 1 1 1
##
## [[12]]
## [1] 1 1
##
## attr(,"mode")
## [1] "binary"
## attr(,"B")
## [1] TRUE
```

Globally standardized

Setting the style argument to `C` gives the so-called globally-standardized weights. This means that the binary weight of each of the neighbor cells is divided by the total number of neighbors. Thus, the globally standardized weights add up to `n`.

```
# C = globally standardized (sums over all links to n)
nb2listw(egII.nb, style = "C")$weights
```

```
## [[1]]
## [1] 0.3529412 0.3529412
##
## [[2]]
## [1] 0.3529412 0.3529412 0.3529412
##
## [[3]]
## [1] 0.3529412 0.3529412 0.3529412
##
## [[4]]
## [1] 0.3529412 0.3529412
##
## [[5]]
## [1] 0.3529412 0.3529412 0.3529412
##
## [[6]]
## [1] 0.3529412 0.3529412 0.3529412 0.3529412
##
## [[7]]
## [1] 0.3529412 0.3529412 0.3529412 0.3529412
##
## [[8]]
## [1] 0.3529412 0.3529412 0.3529412
##
## [[9]]
## [1] 0.3529412 0.3529412
```

```
##
## [[10]]
## [1] 0.3529412 0.3529412 0.3529412
##
## [[11]]
## [1] 0.3529412 0.3529412 0.3529412
##
## [[12]]
## [1] 0.3529412 0.3529412
##
## attr(,"mode")
## [1] "binary"
## attr(,"C")
## [1] TRUE
```

```
sum(unlist(nb2listw(egII.nb, style = "C")$weights))
```

```
## [1] 12
```

In the example above, all the spatial weights sum to 12, which is the sample size, or the total number of cells.

Globally standardized (unity)

Setting the style argument to `U` gives also globally standardized weights, but the spatial weights will now sum to 1. Essentially, the binary weights of each of the neighbor cells is divided by the total number of neighbors. In this example, the total number of neighbors is 34.

```
# U = equal to C divided by n (sums over all links to unity) 1/34
nb2listw(egII.nb, style = "U")$weights
```

```
## [[1]]
## [1] 0.02941176 0.02941176
##
## [[2]]
## [1] 0.02941176 0.02941176 0.02941176
##
## [[3]]
## [1] 0.02941176 0.02941176 0.02941176
##
## [[4]]
## [1] 0.02941176 0.02941176
##
## [[5]]
## [1] 0.02941176 0.02941176 0.02941176
##
## [[6]]
## [1] 0.02941176 0.02941176 0.02941176 0.02941176
##
```

```
## [[7]]
## [1] 0.02941176 0.02941176 0.02941176 0.02941176
##
## [[8]]
## [1] 0.02941176 0.02941176 0.02941176
##
## [[9]]
## [1] 0.02941176 0.02941176
##
## [[10]]
## [1] 0.02941176 0.02941176 0.02941176
##
## [[11]]
## [1] 0.02941176 0.02941176 0.02941176
##
## [[12]]
## [1] 0.02941176 0.02941176
##
## attr(,"mode")
## [1] "binary"
## attr(,"U")
## [1] TRUE
```

```
sum(unlist(nb2listw(egII.nb, style = "U")$weights))
```

```
## [1] 1
```

In the example above, all the spatial weights sum to 1.

Join Tests for Spatial Correlation

In this part, we will be using the small mosaic pattern II from lecture to explore join tests for spatial correlation.

II.

B	B	W	W
B	B	B	W
W	B	W	W

Figure 1: Mosaic Pattern 2

BB, WW, and BW Test by Normal Approximation

Here we consider hypothesis tests for spatial dependence by the join test in R.

The null hypothesis of the test is that the spatial pattern is random (complete spatial randomness) and the alternative hypothesis is that the spatial pattern is non-random.

The function we use is `joincount.test()`, with the following syntax extracted from the function `args()`.

```
args(joincount.test)
```

```
## function (fx, listw, zero.policy = attr(listw, "zero.policy"),
##     alternative = "greater", sampling = "nonfree", spChk = NULL,
##     adjust.n = TRUE)
## NULL
```

Notice that the first argument (`fx`) is a factor and the second argument is a spatial weight object.

To make the numerical vector `egII` a factor, we apply the function `as.factor()`. There are two levels in the output, `"0"` and `"1"`, which indicates that the 0's and 1's are the two possible levels of the factor.

```
levels(as.factor(egII))
```

```
## [1] "0" "1"
```

Recall that the list of spatial weights, `egII.lw` as follows.

```
str(egII.lw)
```

```
## List of 3
##  $ style     : chr "W"
##  $ neighbours:List of 12
```

```
##   ..$ : int [1:2] 2 5
##   ..$ : int [1:3] 1 3 6
##   ..$ : int [1:3] 2 4 7
##   ..$ : int [1:2] 3 8
##   ..$ : int [1:3] 1 6 9
##   ..$ : int [1:4] 2 5 7 10
##   ..$ : int [1:4] 3 6 8 11
##   ..$ : int [1:3] 4 7 12
##   ..$ : int [1:2] 5 10
##   ..$ : int [1:3] 6 9 11
##   ..$ : int [1:3] 7 10 12
##   ..$ : int [1:2] 8 11
##   ..- attr(*, "class")= chr "nb"
##   ..- attr(*, "call")= language cell2nb(nrow = 3, ncol = 4, type = "rook")
##   ..- attr(*, "region.id")= chr [1:12] "1:1" "2:1" "3:1" "4:1" ...
##   ..- attr(*, "cell")= logi TRUE
##   ..- attr(*, "rook")= logi TRUE
##   ..- attr(*, "sym")= logi TRUE
##  $ weights   :List of 12
##   ..$ : num [1:2] 0.5 0.5
##   ..$ : num [1:3] 0.333 0.333 0.333
##   ..$ : num [1:3] 0.333 0.333 0.333
##   ..$ : num [1:2] 0.5 0.5
##   ..$ : num [1:3] 0.333 0.333 0.333
##   ..$ : num [1:4] 0.25 0.25 0.25 0.25
##   ..$ : num [1:4] 0.25 0.25 0.25 0.25
##   ..$ : num [1:3] 0.333 0.333 0.333
##   ..$ : num [1:2] 0.5 0.5
##   ..$ : num [1:3] 0.333 0.333 0.333
##   ..$ : num [1:3] 0.333 0.333 0.333
##   ..$ : num [1:2] 0.5 0.5
##   ..- attr(*, "mode")= chr "binary"
##   ..- attr(*, "W")= logi TRUE
##   ..- attr(*, "comp")=List of 1
##   .. ..$ d: num [1:12] 2 3 3 2 3 4 4 3 2 3 ...
##  - attr(*, "class")= chr [1:2] "listw" "nb"
##  - attr(*, "region.id")= chr [1:12] "1:1" "2:1" "3:1" "4:1" ...
##  - attr(*, "call")= language nb2listw(neighbours = egII.nb)
##  - attr(*, "zero.policy")= logi FALSE
```

Now we apply the join count test to the mosaic pattern II data with a first-order neighborhood by the function `joincount.test()`

```
joincount.test(as.factor(egII), egII.lw, alternative = "two.sided")
```

```
##
##  Join count test under nonfree sampling
##
## data:  as.factor(egII)
```

```
## weights: egII.lw
##
## Std. deviate for 0 = 0.82451, p-value = 0.4097
## alternative hypothesis: two.sided
## sample estimates:
## Same colour statistic           Expectation              Variance
##             1.6666667             1.3636364             0.1350771
##
##
##  Join count test under nonfree sampling
##
## data:  as.factor(egII)
## weights: egII.lw
##
## Std. deviate for 1 = 1.6181, p-value = 0.1056
## alternative hypothesis: two.sided
## sample estimates:
## Same colour statistic           Expectation              Variance
##             1.9583333             1.3636364             0.1350771
```

The output above gives the result of two tests. The first test is a White-White, or W-W test. The observed W-W test statistic is 1.67 with the expectation of 1.36 and a variance of 0.135. Based on a normal approximation, the z-value is 0.82461 and the p-value is 0.4097. We do not reject the null of CSR and there is no evidence that the spatial pattern of the white cells are not CSR.

The second test is a Black-Black, or B-B test. Here, the observed B-B test statistic is 1.96, the expectation is 1.363 and the variance is 0.135. Notice, the expectation and variance are the same for both tests (W-W and B-B). With the B-B test we find the p-value is 0.1056. There is no evidence that the spatial pattern is not CSR.

Recall, W-W and B-B tests are tests of *similarity*. To use the opposite color as in a Black-White, or B-W test, we can use the R function, `joincount.multi()` with the following syntax:

```
# BB, BW and Jtot join count
joincount.multi(as.factor(egII), egII.lw)
```

```
##      Joincount Expected Variance z-value
## 0:0    1.66667  1.36364  0.13508  0.8245
## 1:1    1.95833  1.36364  0.13508  1.6181
## 1:0    2.37500  3.27273  0.49107 -1.2811
## Jtot   2.37500  3.27273  0.49107 -1.2811
```

The W-W (0:0) test and the B-B (1:1) test are the same as those using `joincount.test()`. The B-W test (1:0) gives an observed statistic of 2.375 with an expectation of 3.27273 and variance 0.49107. The z-value is -1.2811. The p-value is not reported, but we can compute that as an exercise.

If this were your own research study, what concerns would you have about drawing conclusions from the 3 by 4 grid? Sample size, for one, would be of concern.

The tests above rely on a normal approximation, but for the normal approximation to be accurate, the grid size needs to be relatively large. A 3x4 grid is quite small and the normality approximation

may not be sufficiently accurate. Thus, we next explore a Monte Carlo version of this test.

BB, WW, and BW Test by Monte Carlo

To perform a Monte Carlo version of the W-W and B-B tests, we use the R function, `joincount.mc()`. We specify the number of Monte Carlo permutations with the argument, `nsim=99`.

We will first set the seed. These tests are based on randomization. However, when you re-run your code, you may not want a different answer every time. By setting the seed, you are telling R to remember what the randomization process was so that it gives you the same answer every time.

```
set.seed(1)
```

We now apply the Monte Carlo join test by the function `joincount.mc()`.

```
joincount.mc(as.factor(egII), egII.lw, nsim = 99)
```

```
##
##  Monte-Carlo simulation of join-count statistic
##
## data:  as.factor(egII)
## weights: egII.lw
## number of simulations + 1: 100
##
## Join-count statistic for 0 = 1.6667, rank of observed statistic = 74,
## p-value = 0.26
## alternative hypothesis: greater
## sample estimates:
##     mean of simulation variance of simulation
##              1.3918350              0.1459722
##
##
##  Monte-Carlo simulation of join-count statistic
##
## data:  as.factor(egII)
## weights: egII.lw
## number of simulations + 1: 100
##
## Join-count statistic for 1 = 1.9583, rank of observed statistic = 94,
## p-value = 0.06
## alternative hypothesis: greater
## sample estimates:
##     mean of simulation variance of simulation
##              1.3564815              0.1608993
```

The Monte Carlo-based test above gives us both the W-W test results and the B-B test results. For the W-W test, the observed test statistic is 1.6667. Since only one-sided test results are reported, we compute the p-value for the two-sided test by multiplying 0.19 by 2 to get a p-value of 0.38. We do not reject the null hypothesis at the 10% level. There is no evidence that the spatial pattern is not CSR.

The B-B Monte Carlo test results have an observed test statistic of 1.9583. The p-value for the one-sided test is 0.02; we multiply 0.02 by 2 to get the p-value for the two-sided test to get a p-value of 0.04. Here, we reject the null hypothesis at the 5% level. We find weak evidence that the spatial pattern is not CSR.